



Assistive Visual AI:

AVA

Image-to-speech scene understanding using YOLO-based detection (yolo26n.pt), SAM 3, BLIP, deterministic language generation, and offline TTS.

Team Members

Aarav · Josh · Artur · Nik

Sight is information — and most AI assumes you have it

~2.2 billion people live with some form of vision impairment globally [8].

Daily tasks rely on continuous visual interpretation: locating obstacles, identifying people, judging spatial layout. Without sight, that situational context collapses.

Modern vision foundation models (YOLO, SAM, BLIP) can compress that interpretation into a few seconds — and deliver it as audio.

2.76 / 5

average user satisfaction
with current AI scene-description apps [9]



Perception gap

Without sight, situational context (what is near, where things are) collapses to touch and audio.



Models exist

Real-time inference of vision foundation models is now possible on consumer hardware.



Speech closes loop

Converting structured scene data into spoken language re-creates that situational context, eyes-free.

Use cases and what we deliberately do not claim

AIM

Build a modular, image-focused proof of concept that converts a visual scene into a short, prioritised, spoken description — and honestly evaluate the trade-off between richness and latency.



Outdoor / street

Bus, people, vehicles, generic obstacles. Tests baseline detection and prioritisation.



Crowded scene

Many detections. Tests filtering: only top-3 of priority-1/2 objects spoken.



Open environment

Sidewalk / café-style scene, mixed object types. Tests robustness.



OUT OF SCOPE — explicit framing

- Live camera streaming from a wearable (we run pre-recorded video instead).
- Real-time deployment claims — the full image pipeline takes ~4 s per frame.
- Depth estimation, mobile / on-device export, and a user study with visually impaired participants.

Where AVA sits

Approach	Strength	Limitation	Our response
Single-shot detection (YOLO family) [1,2]	Fast, real-time-capable	No spatial language, no segmentation	Use as the front-end on both branches
Promptable segmentation (SAM, SAM 3) [3,4]	Pixel-accurate, open-vocabulary	Heavy; slow on consumer hardware	Use, evaluate cost honestly
Vision-language captioning (BLIP) [5]	Natural-language scene gist	Generic; ignores user-relevant priority	Use for global context only
Assistive systems (AIDEN, Seeing AI) [10]	Polished UX, real users	Closed-source or end-to-end LLM-based	Open, modular, audited eval



Where we add value: we integrate detection, segmentation, captioning, prioritisation and speech, then *evaluate the accuracy / latency trade-off* openly — including admitting where the heavy components don't pull their weight.

Existing Products

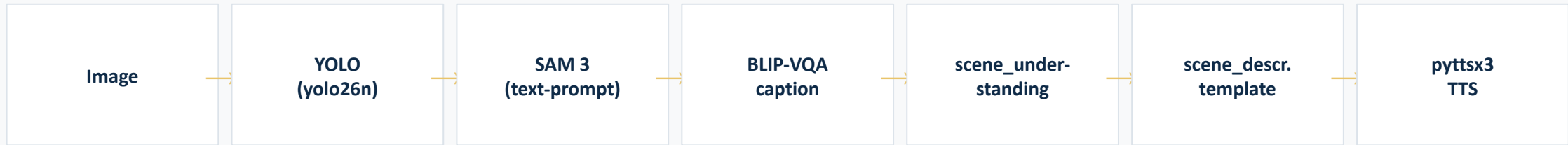
Seeing AI	Be My AI	Google Lookout
Seeing AI- Microsoft Seeing AI visually identifies and describes a number of possible things with multiple models, such as short text, documents, scenes, people, or currency. It only works on still images, however, and needs to be connected to the cloud to perform any processing at all.	Be My AI- Microsoft Uses GPT-4o to describe scenes on user’s computer screens, such as websites or online images. Since it is not intended for real-time use, its latency is unsuitable for assisting the visually impaired in the outside world. Its usage of only an LLM also introduces the risk of hallucinations into the descriptions	Google Lookout- Google The closest to our project, this is an Android app that continuously receives a camera feed and verbally announces prominent objects in frame. It is run locally for speed and privacy. Does not allow for the user to customise the announcements or detections to their preferences. Since it is run on the user’s phone, it may prevent visually impaired users from accessing other tools they would need on their device.

Where Our Product Differs

Our product would ideally be run from dedicated hardware, such as a lapel camera or technology similar to Meta’s Meta Glasses, rather than having to use hardware users may already be using. We would also offer customisation settings to improve user experience, such as how often the product describes a scene and how many items it will describe at once.

Two pipelines, one back end

IMAGE BRANCH · Full pipeline · ~4.0 s / frame



VIDEO BRANCH · Lightweight pipeline · ~25 ms / frame



WHY TWO PIPELINES — IT IS A REAL ARCHITECTURAL CHOICE IN `main.py`

- Image branch is for *single rich descriptions* — masked image fed to BLIP for global context, segmentation drives positions.
 - Video branch is the *fast deployment-realistic path* — no SAM, no BLIP, runs framewise.
- Both branches share the back-end (templates + TTS).

Object detection — YOLO-based (yolo26n.pt)

Why YOLO?

Single-shot — one forward pass per frame. Latency dominates the user experience: a slow caption is a useless caption.

Mature — pretrained on COCO (80 classes), well-aligned with the everyday objects a mobility-impaired user encounters.

Edge-friendly — the nano-scale weights (~5 MB) are deployable on consumer hardware. Runs in ~25 ms / image in our 100-image evaluation.

In our code

Model file: yolo26n.pt (an Ultralytics YOLO checkpoint shipped with the repo)

Loaded via: ultralytics.YOLO("yolo26n.pt")

Confidence floor: 0.4 (used in both branches before downstream stages)

OUTPUT SCHEMA · vision/object_detection.py

```
def get_bounding_boxes_from_frame(
    frame, model, class_names, debug):
    results = model(frame, ...)
    return [
        { "object":      "person",
          "box":         [x, y, w, h],
          "confidence": 0.91 },
        { "object":      "bus",
          "box":         [x, y, w, h],
          "confidence": 0.86 },
        ...
    ]
```

Segmentation — SAM 3

Why SAM3?

Concept-prompted segmentation, not box-prompted

We use facebook/sam3 via Hugging Face. Instead of prompting SAM with YOLO's bounding boxes, we prompt with the unique class labels YOLO returns — "person", "bus", "chair".

SAM 3 then finds masks for those concepts *across the whole image*, giving us pixel-level evidence for downstream evaluation against COCO ground truth.

CONFIG · vision/SAM3_seg.py

```
Sam3Model.from_pretrained("facebook/sam3")
threshold=0.5, mask_threshold=0.5
confidence_threshold=0.4 (matches YOLO)
```



HONEST TRADE-OFF

SAM 3 dominates ~94.5% of our pipeline runtime.

We keep it in the image branch because it gives us the only *pixel-level evidence* in the system, which lets us run mask IoU against COCO. We *do not* use it on the video branch - that's the whole point of having two pipelines.

BLIP — global scene caption

Why BLIP?

BLIP-VQA used as a caption-completer

Salesforce/blip-vqa-base — the visual-question-answering checkpoint, prompted with "a photo of". We exploit the VQA interface as a constrained caption-completer, outputs are short and topical.

BLIP receives the SAM-mask-overlaid image, so the caption is grounded on the segmented foreground.

Chosen as it is computationally light and does not have API or cloud requirements. (necessary for dedicated hardware, such as a lapel camera or technology similar to Meta's Meta Glasses, which AVA would be run on)

Implemented but not in default path

vision/BLIP2_scene.py exposes blip2-flan-t5-xl as a heavier captioning alternative - better quality, large download, slower. *Not wired into main.py by default.*

WORKED EXAMPLE

BLIP caption (global)

"A bus parked on a street."

+ Object sentence (spatial)

"There are two persons in the centre, and a bus on the left."

= Final spoken (concatenated)

"A bus parked on a street. There are two persons in the centre, and a bus on the left."

Scene understanding → deterministic speech

The detector returns dozens of objects. A useful sentence contains three. This stage is the bridge — and the project's main contribution beyond integration.

PRIORITY · vision/scene_understanding.py

```
PRIORITY = {
    "person": 3,    # always speak
    "car": 3,
    "bus": 3,
    "bicycle": 3,
    "dog": 2,      # often speak
    "cat": 2,
    "chair": 1,    # if prominent
    "table": 1,
    "tv": 1,
    # default unknown -> 1
}
# main.py: filter_threshold = 3
```

1. Position

Centroid x → left / centre / right (split into thirds of frame width).

2. Filter

All priority-3 kept. Top-k of priority-1/2 by priority (k = 3, hard-coded, but easily made user adjustable). Deduplication on (object, position).

3. Sentence

Deterministic template via inflect (a/an, plurals). Falls back to “Nothing to see here.” when empty.



Audio: pyttsx3 (offline, OS-level), threaded so the GUI never blocks.

Why no LLM for description: no hallucination, deterministic latency (~0.5 ms), no API dependency, auditable.

Modular code, by responsibility

Orchestration

`main.py` – Tkinter GUI, threading, mode dispatch, debug toggle.

Vision

`object_detection.py` · `SAM3_seg` · `BLIP_scene` · `BLIP2_scene` · `scene_understanding`.

Language

`scene_description.py` – template engine + grammar (inflect).

Audio

`tts.py` – pyttsx3 wrapper, offline.

Evaluation

`iou_eval` · `description_eval` · `latency_eval` · `main_eval` (writes CSV summary).

DESIGN PROPERTIES

Modular

Each pipeline stage in one file — swap any model independently.

Reproducible

COCO 100-image subset shipped in `coco_100/`. Eval re-runnable from clean clone.

Two paths

Image and video branches share the language + audio back-end.

Documented

README + requirements.txt; debug mode prints all intermediate artefacts.

What we measure, and on what data

Dataset used for Evaluation	METRIC 01 · PERCEPTION	METRIC 02 · EXPRESSION	METRIC 03 · DEPLOYABILITY
COCO Dataset The COCO dataset includes over 330,000 images with 80 object categories. COCO is considered the “gold standard” for benchmarking computer vision models [11]. It is widely used by researchers and practitioners to train models like YOLO. SIZE USED 100 Images - Due to evaluation time DATASET COCO val2017 [6]	Mask IoU Predicted SAM 3 masks vs COCO ground-truth masks. Per-image, per-class best-match IoU. In order to allow for a fair comparison, objects which “missed” due to the priority rules are excluded from the IoU measurement as it would misinform the accuracy due to logical pipeline decisions OUTPUT iou_eval.csv DATASET COCO val2017 [6]	Object-caption semantic overlap BERTScore [7] between SAM-detected unique object names ("person bus chair") and COCO captions. As SAM3 outputs a list of objects while COCO has natural sentences, BERTScore Precision was used to measure the semantic relevance of object detections while avoiding penalizing the priority-based filtered outputs, rather than exact word matching OUTPUT description_eval.csv DATASET COCO captions [6]	Latency Per-stage wall-clock for YOLO, SAM 3, BLIP, language; plus end-to-end total per image. Evaluation Infrastructure affects values. Details about the infrastructure used for this evaluation can be found on the next slide. OUTPUT latency_eval.csv DATASET COCO val2017 [6]

[6] Lin et al., COCO, arXiv:1405.0312 · [7] Zhang et al., BERTScore, ICLR 2020, arXiv:1904.09675
COM3025 · Assistive AI for Visually Impaired Users

Headline numbers — 100 COCO val2017 images

Evaluation Infrastructure

GPU - RTX 3060 Mobile

VRAM - 6GB

CPU - Intel i7 11800H

RAM - 16GB

0.576

MEAN MASK IoU

654 matched predictions · 22 unmatched

0.438

OBJECT-CAPTION OVERLAP Precision

BERTScore vs COCO captions, n = 100

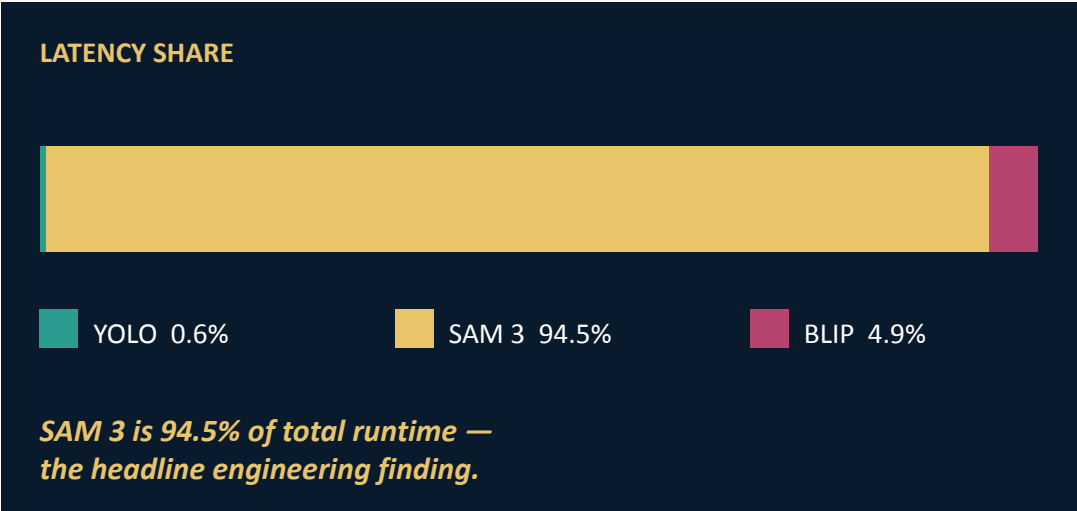
3.99 s

MEAN END-TO-END LATENCY

image branch, full pipeline, n = 100

PER-STAGE LATENCY · mean per image

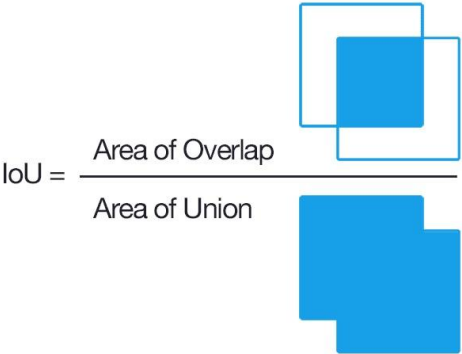
Stage	Time (s)	Share
YOLO (yolo26n)	0.025	0.6%
SAM 3	3.774	94.5%
BLIP (vqa-base)	0.195	4.9%
Language (template)	0.00049	<0.1%
Total	3.994	100%



What the numbers actually mean

KEY FINDINGS

- **Mean IoU 0.576** = SAM lands on roughly the right region but is not pixel-perfect.
 - **Latency: SAM 3 = 94.5%** of total runtime, the single dominant cost.
 - **YOLO 25 ms + template 0.5 ms** → the lightweight branch is already practical for live framewise use.
 - **Object–caption Precision 0.438** is structurally low: short object lists vs full COCO captions. It is a content-overlap floor, not a quality verdict. The set-up is a custom string level comparison of the labelled caption stripped with connectors with the aim of only including objects and comparing with the output string of objects using BERTScore semantic similarity.
- This causes some limitations as the labelled captions may have not been stripped completely of non-object words and may have included synonyms or choice of objects to include within caption.*



(Source: Rosebrock, 2016)

THE ENGINEERING CONCLUSION

Richer perception comes at a latency cost the assistive context cannot ignore.

	Lightweight (video)	Full (image)
Latency	≈ 25 ms	≈ 4 s
Spatial detail	bbbox thirds	pixel masks
Caption	template only	BLIP + template
Best for	live frames	rich one-off

→ Recommendation: use the lightweight branch as the live path; reserve the full pipeline for one-off scene queries.

Live walkthrough plan

0:00	<code>python main.py</code>	Tkinter GUI appears — three buttons: Image / Video / Debug
0:30	Toggle Debug ON	Debug mode label updates — examiner will see all intermediate prints
0:45	Image → <code>test_pics/bus.jpg</code>	YOLO box window + SAM mask overlay + console JSON + final sentence + audio
2:00	Image → <code>test_pics/crowd.jpg</code>	Demonstrates prioritisation under noise — top-3 of priority-1/2 spoken
3:15	Video → <code>city_walk.mp4, wait = 5 s</code>	Lightweight branch live: framewise YOLO + template + audio at each interval
4:30	<code>cat evaluation/results/main_eval_summary.csv</code>	Closes the loop between the deck and the actual code

Limitations and what would change next

LIMITATIONS — honest

- **SAM 3 is the single dominant cost.** Image branch is not real-time on consumer hardware.
- **No depth estimation.** Bounding-box area conflates small-far with small-close.
- **COCO classes miss assistive-critical objects** — kerbs, stairs, pedestrian crossings, doorways are not detected.
- **Object-caption overlap underrates assistive style** by construction. COCO captions are the wrong reference for spatial guidance.
- **No user study** with visually impaired participants — usability is unmeasured [9].

FUTURE WORK — concrete

- **Lighter mask model** (FastSAM, MobileSAM) — or skip segmentation on the live path entirely.
- **Monocular depth** (e.g. Depth Anything v2) for real near/far disambiguation.
- **Object tracker** so the same object isn't redescribed every wait-cycle on the video branch if the position does not change.
- **Open-vocabulary detector** (YOLOE / SAM 3 with custom prompts) for assistive-critical classes.
- **ONNX / CoreML export** for mobile and on-device deployment.
- **User study** with assistive-utility ratings rather than COCO captions as reference.

Built · Evaluated · Learned

BUILT

A modular image-to-speech assistive prototype

Two pipelines (image / video) sharing a templated language + offline TTS back-end across 9 modules.

EVALUATED

On 100 COCO val2017 images, three independent metrics

Mean mask IoU 0.58 · object–caption overlap (BERTScore F1) 0.27 · end-to-end latency 4.0 s (image) / 25 ms (video).

LEARNED

Assistive AI is a balance of perception and time

SAM 3 is the dominant cost. The lightweight branch is the deployment-realistic path. Honest evaluation > inflated claims.

INDIVIDUAL CONTRIBUTIONS		
Member	Module	%
Aarav	Text-to-speech + Presentation	25%
Nikita Izbrodin	Object detection + GUI + video/image pipeline	25%
Artur	Evaluation + Analysis	25%
Josh	SAM3 + BLIP + scene understanding	25%

Detailed contribution statement included in the submission package.

References

- [1] J. Redmon, S. Divvala, R. Girshick, A. Farhadi. "You Only Look Once: Unified, Real-Time Object Detection." arXiv:1506.02640, 2016.
- [2] R. Sapkota, M. Karkee. "Ultralytics YOLO Evolution: An Overview of YOLO26, YOLO11, YOLOv8 and YOLOv5." arXiv:2510.09653, 2025.
- [3] A. Kirillov et al. "Segment Anything." arXiv:2304.02643, 2023.
- [4] N. Carion, L. Gustafson et al. "SAM 3: Segment Anything with Concepts." arXiv:2511.16719, 2025.
- [5] J. Li, D. Li, C. Xiong, S. Hoi. "BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation." arXiv:2201.12086, 2022.
- [6] T-Y. Lin et al. "Microsoft COCO: Common Objects in Context." ECCV 2014. arXiv:1405.0312.
- [7] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, Y. Artzi. "BERTScore: Evaluating Text Generation with BERT." ICLR 2020. arXiv:1904.09675.
- [8] World Health Organization. "World Report on Vision." 2019. <https://www.who.int/publications/i/item/9789241516570>
- [9] A. Stangl et al. "Investigating Use Cases of AI-Powered Scene Description Applications for Blind and Low Vision People." CHI '24. arXiv:2403.15604.
- [10] L. Marquez-Carpintero et al. "AIDEN: Design and Pilot Study of an AI Assistant for the Visually Impaired." arXiv:2511.06080, 2025.
- [11] T-Y. Lin et al., "Microsoft COCO: Common Objects in Context," ECCV, 2014. Available: <https://arxiv.org/abs/1405.0312>
- [12] Rosebrock, A. (2016) 'Intersection over Union (IoU) for object detection', PyImageSearch, 7 November. Available at: <https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/> (Accessed: 4 May 2026).